

Engineering Autonomous Trust: A Formal Decision Firewall, Governance, Audit, and Observability Framework for Fast Probabilistic Decision Models

Operational Risk Topology of Constrained Decision Models

The deployment of constrained, non-autoregressive decision models marks a fundamental architectural departure from generative artificial intelligence¹. Whereas conversational Large Language Models (LLMs) sequentially sample text tokens across broad probability spaces, specialized System One engines—exemplified by TypeSafe AI's Jev—evaluate unstructured state against pre-declared, typed questions in a single parallel inference pass². By eliminating natural-language generation, these engines restrict outputs entirely to typed primitives, such as categorical selections (Choice), ordinal rankings (Score), and binary probabilities (Noul)².

Operating with latencies between 70 and 500 milliseconds at fractional-cent cost structures, these models are deliberately engineered for high-frequency operational execution, effectively serving as probabilistic branching logic directly embedded within software runtimes².

While this design guarantees syntactic validity and eliminates open-ended linguistic hallucinations, it introduces a more dangerous operational vulnerability: the silent, uninspected automation of consequential actions¹. When an autoregressive LLM misjudges a prompt, the error manifests visually within the generated prose, exposing a textual surface that human operators can challenge¹. In contrast, a constrained decision model emits a bare probability or category that is consumed directly by programmatic application code¹. Because the decision operates beneath user interfaces, individuals affected by the system experience only its downstream operational consequences, entirely unaware of the underlying model, its alternative options, or its statistical certainty¹.

A fundamental defect in this architecture is the conflation of calibration with explainability¹. Although training methodologies like Reinforcement Learning for Calibrated Decisions (RLCD) optimize predictions to reflect empirical event frequencies across synthetic or historical datasets, a calibrated probability remains a statistical summary rather than a causal justification¹. A decision payload indicating an 85% probability of loan default reveals the model's confidence, but it fails to communicate which evidence determined the judgment, which trade-offs were negotiated, whether unverified inferences displaced observed facts, or how close an alternative candidate was to selection¹.

This lack of internal transparency enables *incentive laundering*, wherein commercial preferences, margin optimizations, or cost-minimization objectives are converted into seemingly

neutral, calibrated probabilities¹. In routing, underwriting, or procurement workflows, a decision engine can systematically prioritize sponsored vendors, high-margin financial products, or lower-cost support tiers while preserving the mathematical appearance of objective judgment¹. If challenged, operators can appeal to statistical uncertainty and multi-factor complexity to obscure deliberate commercial skew¹.

Systemic Failure Mode	Operational Mechanism	Manifest Liability & Impact
Incentive Laundering	Latent optimization of commercial preferences embedded within prompt context, loss functions, or fine-tuning datasets ¹ .	Algorithmic preference for sponsored or high-margin outcomes under the guise of neutral probabilistic scoring ¹ .
Decision Monoculture	Sector-wide deployment of an identical base decision model across independent, competing institutions ¹ .	Synchronized systemic failures, correlated credit or supply shocks, and uniform blind spots across distributed networks ¹ .
Objective Drift	Latent shifts in enterprise operational goals while input schemas, system prompts, and threshold gates remain static ¹ .	Autonomous optimization of outdated corporate priorities without the generation of explicit software errors ¹ .
Threshold Laundering	Programmatic alterations to downstream execution thresholds without corresponding adjustments to model configuration ¹ .	Silent elevation of organizational risk profiles disguised as uniform model behavior ¹ .
Context Poisoning	Adversarial or commercially biased state representations injected by upstream data collectors or agentic planners ¹ .	Generation of corrupt, discriminatory, or unauthorized outputs despite strict schema validation ¹ .
Selective Invocation	Upstream programmatic branching that routes contexts to the model only	Evasion of governance frameworks and audit logs through intentional sampling

	when favorable outcomes are anticipated ¹ .	bias ¹ .
Automation Asymmetry	Autonomous execution of profitable or adverse outcomes paired with manual, friction-heavy queues for user-favorable actions ¹ .	Systematic unfairness, extended processing delays for consumer remedies, and disparate regulatory impact ¹ .
Appeal Blindness	Optimization of model weights exclusively on accepted decisions, leaving rejected cohorts completely unobserved ¹ .	Compounding historical biases due to the structural absence of counterfactual outcome data ¹ .
Proxy Discrimination	Unregulated contextual state (e.g., zip codes, browser configurations) acting as mathematical surrogates for protected demographics ¹ .	Unlawful discriminatory patterns emerging within fully typed, compliant output spaces ¹ .
Responsibility Diffusion	Fragmented orchestration where responsibility is dispersed across model developers, prompt engineers, and human reviewers ¹ .	Complete dissolution of legal accountability across enterprise operational boundaries ¹ .

Addressing these systemic vulnerabilities requires recognizing that schema compliance does not equal decision safety¹. Enterprises deploying engines like Jev within operational workflows must construct an external, vendor-neutral control plane that treats model outputs as unverified proposals requiring deterministic policy validation, cryptographic receipting, counterfactual verification, and continuous outcome tracking¹.

The Inline Decision Firewall and Runtime Gatekeeper

To prevent uninspected probabilistic outputs from executing unauthorized system actions, an inline Decision Firewall must intercept candidate choices prior to downstream dispatch¹. This architecture formalizes the boundary between the probabilistic judgment of the AI model and the deterministic Policy Enforcement Point (PEP) and Policy Decision Point (PDP) of the host application, guaranteeing that machine learning outputs cannot independently initiate external side effects⁶.

The firewall mediates execution by calculating a composite Execution Risk Score (R) for every proposed action¹. Model confidence cannot serve as the sole governance metric; high confidence in a catastrophic or unlawful action magnifies operational danger rather than mitigating it¹. The firewall synthesizes risk across four core vectors¹:

$$R = I \times U \times V \times A$$

Impact ($I \in [0, 10]$) reflects the real-world consequence and irreversibility of the action, assigning high values to financial disbursements, credential revocations, or adverse customer notifications¹. Uncertainty ($U \in [0, 1]$) represents the complement of model confidence adjusted for historical calibration divergence, expressed as $U = 1 - c$ ¹. Vulnerability ($V \in [1, 5]$) establishes the susceptibility of the operational context to incentive laundering, prompt injection, or commercial bias¹. Autonomy ($A \in [1, 5]$) quantifies the immediacy with which the action impacts external state in the absence of human intervention¹. The quantitative score dictates dynamic routing through a tri-state gate¹. Proposed actions with risk scores falling below a low-risk threshold (τ_{low}) proceed directly to deterministic policy verification for autonomous dispatch¹. Scores between the low-risk and high-risk thresholds ($\tau_{\text{low}} \leq R < \tau_{\text{high}}$) enter an automated evidence challenge mode, pausing dispatch until required missing data is retrieved or counterfactual tests pass¹. Proposed actions that breach the high-risk ceiling ($R \geq$) are blocked from autonomous execution and routed into an asynchronous review pipeline requiring multi-stakeholder or committee sign-off¹. Once an action satisfies risk gating, it is evaluated by a deterministic policy engine embedded within the gatekeeper's hot path⁸. The AWS Cedar policy language provides sub-millisecond evaluation latencies, a loop-free grammar that guarantees termination, and formal mathematical verification developed in the Lean interactive theorem prover⁹. Cedar enforces a strict default-deny posture and order-independent evaluation, ensuring that permissions cannot be granted inadvertently through sequencing errors⁹.

Code snippet

```
from aws_cedar_policy import Policy, Governed, MultiStep, Vendor, Disbursement, ...
```

```

    action == Action: "execute_disbursement"
    resource
  |
  when
    context.execution_risk_score > 40.0
    context.commercial_conflict_declared == true
  |

  permit
  principal == DecisionEngine: "dex_v1_13"
  action == Action: "execute_disbursement"
  resource in Resource.Type: "VendorAccount"
  |
  when
    context.calibrated_confidence >= 0.85 &&
    context.disbursement_amount <= principal.delegated_limit &&
    context.evidence.provenance.containsAll(["invoice_verified", "tax_compliance_valid"])
  &&
  context.evidence.provenance.contains("missing_vendor_attestation")
  |

```

By decoupling the policy layer from internal application logic, security and compliance teams retain central administrative authority over operational boundaries without requiring model redeployment⁹. Downstream systems reject any instruction lacking a cryptographically valid capability token issued directly by the firewall upon successful policy evaluation⁶.

Operational Characteristic	Formally Verified Engine (AWS Cedar)	General Policy Engine (OPA / Rego)	Procedural Application Logic
Mathematical Soundness	Formally verified via Lean theorem prover; provable absence of contradictory bypass paths ⁹ .	Unverified; correctness depends entirely on manual policy test coverage ⁹ .	Unverified; highly susceptible to programmatic regressions and runtime logic bugs ⁹ .
Evaluation Latency	Sub-millisecond, in-process execution suitable for high-throughput	Low-millisecond; commonly implemented via external sidecars or	Sub-microsecond; executed directly within host language instruction

	decision hot paths ⁹ .	local REST daemons ⁹ .	streams.
Default Security Posture	Default-deny; permit granted only through explicit, unambiguous rule satisfaction ¹³ .	User-configured default; policy evaluation order dictates permission outcomes.	Ad-hoc; varies according to individual software engineering implementations.
Static Verification	Compile-time analysis identifies unreachable rules and contradictory security policies ⁹ .	Syntactic linting and schema validation without formal safety proofs.	Static analysis tools (AST inspection) limited to code structure rather than security semantics.
Agent Sidecar Ecosystem	Deep native integration with AWS Bedrock AgentCore and emerging agent frameworks like OpenLeash ¹⁰ .	Standardized adoption across cloud-native infrastructure, Kubernetes, and service meshes ⁹ .	Monolithic and proprietary to specific application codebases.

Cryptographic Decision Receipts and Evidence Provenance Engine

Auditing autonomous decisions through the unstructured capture of model prompts, textual responses, or internal reasoning traces is ineffective¹. Post-hoc explanations generated by AI architectures represent subjective approximations rather than verifiable accounts of operational causes¹. Furthermore, retaining uncured contextual text introduces severe data privacy liabilities by capturing sensitive or non-compliant information within long-term storage⁶.

A defensible audit architecture replaces conversational logging with structured, immutable Decision Receipts, aligning with the IETF draft-farley-acta-signed-receipts specification and the in-toto supply chain attestation standard⁸. Under this framework, evidence provenance must explicitly distinguish between six distinct contextual categories: observed enterprise facts validated by external oracles, probabilistic inferences produced by upstream models, explicitly identified missing information, active policy digests, declared organizational objectives, and commercial disclosures such as vendor rebates or promotional partnerships¹.

Decision receipts achieve cryptographic non-repudiation by serializing all context, inputs, and gatekeeper evaluations using the deterministic JSON Canonicalization Scheme (RFC 8785)⁸. The canonical payload is signed with an Ed25519 private key stored in a Hardware Security

Module (HSM)⁸. Receipts are linked into tamper-evident hash chains by embedding the SHA-256 digest of the immediately preceding receipt (previous_receipt_hash) into each subsequent record¹⁶. To guard against local data manipulation or retroactive log alterations, the root of the local receipt Merkle tree is periodically anchored to an append-only transparency log such as Sigstore Rekor¹⁶.

JSON

```
{
  "_type": "https://in-toto.io/Statement/v1",
  "receipt_id": "rcpt_6a1f8e24_c903_4e81_a5b7_1982dcfa8201",
  "timestamp": "2026-09-22T14:32:01.104Z",
  "previous_receipt_hash": "sha256:d8c412...b910",
  "subject": {
    "name": "BuzzDecisionPipeline:DisbursementAuthorization",
    "digest": "sha256" "8f32a1...bc44"
  },
  "predicateType": "https://typesafe.ai/acta/decision-receipt/v1",
  "predicate": {
    "decision_context": {
      "model_identifier": "typesafe-ai/jev-1.13.0",
      "declared_objective": "minimize_disbursement_latency",
      "delegated_authority_limit": 50000.00,
      "commercial_relationships": {
        "vendor_id": "logistics_partner_alpha",
        "relationship_tier": "tier_1_preferred",
        "commercial_rebate_pct": 0.025
      }
    },
    "evidence_provenance": {
      "dimension": "observed_facts",
      "state": "satisfied",
      "source": "authority" "erp:oracle_sap" "ref" "sha256:1a8b...01e"
    },
    "dimension": "missing_information",
    "state": "unverified"
  }
}
```

```

"source" "authority" "vendor_compliance" "ref" "soc2_type2_cert"
"model_evaluation"
"primitive" "Choice"
"candidates" "vendor_alpha" "vendor_beta" "vendor_gamma"
"selected_action" "vendor_alpha"
"probability_distribution"
  "vendor_alpha" 0.76
  "vendor_beta" 0.14
  "vendor_gamma" 0.10
"calibrated_confidence" 0.83
"gate_enforcement"
  "engine" "AWS_Cedar_v4_13"
  "policy_digest" "sha256:f7d4e1...29ca"
  "execution_risk_score" 34.2
  "gate_decision" "ALLOW"
  "capability_token" "paseto.v4.public.eyJ...88a"
"signatures"
  "keyid" "hsm-sec-gw-01"
  "sig" "3045022100...fa2"

```

The receipt contains no proprietary prompt text or unprotected consumer identifiers, ensuring that sensitive data is preserved within authorized transactional systems while the receipt remains safe for regulatory inspection⁶. Downstream systems verify the capability token offline using public key cryptography before executing actions, guaranteeing that state transitions cannot occur without a cryptographically bound decision record⁶.

Runtime Verification: Counterfactual Sensitivity and Multi-Judge Consensus

Because non-autoregressive decision models emit structured probabilities without natural-language rationales, causal verification must be achieved through empirical perturbation and multi-model consensus rather than subjective dialogue¹. Interrogating a decision system after execution encourages post-hoc confabulation; true causal explainability requires

determining the exact conditions under which an action alters or reverses¹.

Counterfactual sensitivity testing treats the decision model as an opaque mathematical operator, systematically perturbing input vectors to establish the boundaries that invert selected outcomes¹. The runtime firewall executes these tests dynamically across three primary dimensions:

Commercial decoupling isolates potential incentive laundering by stripping commercial identifiers, partnership tiers, and rebate variables from the operational state¹. If removing commercial preferences flips the selected choice toward a non-sponsored alternative, the decision is flagged for high incentive sensitivity, immediately terminating autonomous execution and routing the transaction to human oversight¹.

Demographic perturbation assesses whether decisions hinge on contextual proxies for protected characteristics¹. The firewall programmatically substitutes geographic markers, institutional identifiers, or regional behavioral patterns with neutral baselines¹. A decision flip under neutral demographic substitution indicates unacceptable proxy discrimination, triggering policy rejection regardless of model confidence¹.

Evidence ablation and imputation evaluates the stability of decisions formed under incomplete information¹. When missing information is detected in an adverse decision, the firewall evaluates the model across conservative, median, and optimistic values for the absent variables¹. If an optimistic imputation changes an adverse judgment into an approval, the system suspends execution and issues an automated data-request task, ensuring that individuals are not penalized for verifiable omissions¹.

To mitigate the systemic risks of decision monoculture, consequential actions must never rely solely on a single model provider¹. The architecture implements a tiered consensus protocol, combining fast System One models with deterministic rules and independent shadow judges¹.

Consensus Tier	Functional Role	Technology Foundation	Core Governance Mandate
Primary Evaluator	System One Decision Engine	TypeSafe AI Jev (jev-1.13.0) ²	High-frequency inference, structured schema parsing, and candidate generation ² .
Deterministic Guard	Policy Gatekeeper	AWS Cedar Policy Engine ⁸	Non-negotiable regulatory compliance, structural bounds, and authorization

			limits ⁹ .
Independent Shadow Judge	Cross-Lineage Evaluator	Independent Open-Weight Foundation Model	Parallel asynchronous verification trained on independent data to detect shared blind spots ¹ .
Consensus Arbiter	Disagreement Resolver	Consensus Engine (Aikmat Layer) ¹	Quantitative scoring of inter-model divergence, authority weighting, and human escalation ¹ .

In this consensus pipeline, an action proposal from the primary decision model is cross-referenced against the independent shadow judge. If the primary model approves an action with high confidence while the shadow judge rejects it, the consensus engine calculates a high divergence metric. This divergence supersedes model confidence, causing the firewall to block autonomous execution and route the conflicting evaluations to a human supervisor.

Consequence-Aware Observability and Post-Execution Audit

Standard machine learning observability infrastructure measures operational health via technical telemetry—such as request volume, token latency, and input drift¹. In autonomous decision workflows, these metrics fail to capture real-world operational harm¹. A decision model can exhibit perfect uptime, low latency, and zero schema violations while systematically driving regulatory non-compliance, discriminatory outcomes, or severe financial losses¹.

Decision observability must therefore operate as a closed-loop system connecting initial intent, model choice, gate permission, and downstream consequence⁶.

A central element of this observability is the continuous decomposition of model accuracy against real-world ground truth¹. Model calibration drifts as real-world distributions evolve¹. The observability framework evaluates empirical outcomes using the Murphy decomposition of the Brier score, isolating reliability, resolution, and uncertainty²³:

$$BS = \frac{1}{N} \sum_{i=1}^N (y_i - p_i)^2 = \text{REL} - \text{RES} + \text{UNC}$$

When predictions are grouped into K reliability bins of size n_k , with average predicted

probability $\bar{p}_k$ and observed empirical frequency $\bar{o}_k$, the score decomposes as²³:

$$\text{REL} = \frac{1}{N} \sum_{k=1}^K n_k (\bar{p}_k - \bar{o}_k)^2$$

$$\text{RES} = \frac{1}{N} \sum_{k=1}^K n_k (\bar{o}_k - \bar{y})^2$$

$$\text{UNC} = \bar{y}(1 - \bar{y})$$

Reliability (**REL**) isolates calibration error, measuring the degree to which predicted probabilities match empirical outcomes; perfect calibration yields a reliability penalty of zero²³.

Resolution (**RES**) assesses the model's ability to separate distinct outcomes, rewarding forecasters that assign sharply differentiated probabilities rather than reverting to the baseline rate²³. Uncertainty (**UNC**) captures the irreducible entropy of the underlying environment, completely decoupled from model performance²³.

To ensure statistical precision across non-uniform workloads, the observability engine pairs the Murphy decomposition with the Adaptive Calibration Error (ACE)²⁶:

$$\text{ACE} = \frac{1}{K} \sum_{k=1}^K |\text{acc}(b_k) - \text{conf}(b_k)|$$

Unlike standard Expected Calibration Error (ECE)—which relies on uniform-width bins that group sparse, high-risk predictions into broad buckets—ACE constructs dynamic bins containing equal sample volumes²⁶. This prevents severe calibration errors in the critical tails of decision distributions from being hidden by dense regions near the center²⁶.

Alongside statistical calibration, the observability framework monitors the distribution of real-world outcomes across six continuous operational telemetry streams:

Beneficiary tracking analyzes the commercial impacts of model choices over rolling time windows, evaluating whether selected alternatives disproportionately favor internal monetization streams or preferred vendor tiers¹. Any deviation between stated enterprise objectives and actual economic beneficiaries triggers automated audit reviews¹.

Appeal and escalation ingestion captures formal customer disputes, service tickets, and legal challenges, linking them directly to the originating decision receipt identifier¹. Spikes in customer

challenges within specific decision schemas provide early warning of operational failure prior to macro-level performance degradation¹.

Human override directionality measures the frequency and nature of human supervisor interventions within escalation queues¹. The observability layer separates overrides into favorable and adverse trajectories; an asymmetric operational pattern—where operators routinely confirm automated denials but overturn approvals—signals operational bias in human oversight¹.

Counterfactual outcome integration resolves "appeal blindness" by incorporating external ground-truth datasets, such as third-party credit bureau updates, to evaluate rejected applicants who secured financing elsewhere¹. This ensures the model's training pipeline is not permanently biased by the absence of negative outcome data¹.

Impact drift monitoring continuously measures disparate impact ratios across protected groups, ensuring that shifts in contextual state do not introduce systemic bias¹.

Execution gap analysis reconciles authorized capability tokens with physical transaction logs to identify effector failures, partial network drops, or unauthorized command executions⁶.

Regulatory Compliance and Legal Defensibility

Operating autonomous decision systems without verifiable governance layers exposes organizations to significant legal liabilities under emerging global regulatory standards¹⁵.

Regulatory bodies increasingly reject algorithmic obscurity, establishing binding mandates for traceability, bias mitigation, and human oversight¹⁵.

The European Union AI Act (Regulation EU 2024/1689) imposes strict legal requirements on models deployed within high-risk domains, including employment, credit evaluation, healthcare, and public infrastructure¹. Article 12 mandates continuous, automated logging over the entire operational lifecycle to guarantee technical traceability¹. The cryptographically signed decision receipts fulfill this requirement by preserving an immutable, replayable audit trail anchored in public transparency logs¹. Article 14 requires that high-risk systems maintain mechanisms enabling human oversight to minimize automation bias and provide immediate operational override capabilities¹. The firewall's tri-state gating mechanism fulfills this mandate by routing high-risk decisions into human-in-the-loop workflows prior to execution¹. Article 15 mandates technical robustness and cybersecurity resilience against adversarial manipulation, which the framework satisfies through multi-model consensus and counterfactual input testing¹.

In the United States, Consumer Financial Protection Bureau (CFPB) Circulars 2022-03 and 2023-03 eliminate the defense of algorithmic complexity under the Equal Credit Opportunity Act (ECOA) and Regulation B (12 CFR Part 1002)¹⁵. Creditors are legally prohibited from asserting that the uninterpretable or "black-box" nature of an AI model excuses them from providing consumers with the specific, accurate reasons for an adverse credit action¹⁵. Furthermore, Circular 2023-03 clarifies that relying on generalized checklist forms does not satisfy statutory obligations if the underlying model scored alternative or non-traditional behavioral data⁷. The Decision Firewall addresses these mandates by isolating observed facts from inferences and linking adverse determinations directly to missing or insufficient evidence, enabling the

automatic generation of compliant adverse action notices¹.
The National Institute of Standards and Technology (NIST) AI Risk Management Framework (AI RMF 1.0) and its Generative AI Profile (NIST AI 600-1) provide organizational blueprints across four functions: Govern, Map, Measure, and Manage¹. The framework maps directly to these requirements: Govern establishes accountability and separation of duties; Map catalogs operational context, dependencies, and risks; Measure quantifies calibration error and disparate impact; and Manage enforces runtime technical controls via the inline firewall¹.

Legal & Regulatory Standard	Statutory / Administrative Mandate	Decision Firewall Implementation Mechanism
EU AI Act: Article 12	Continuous, automated event logging to ensure system traceability throughout lifecycle ¹ .	Canonical Ed25519-signed decision receipts anchored to Sigstore Rekor transparency logs ⁸ .
EU AI Act: Article 14	Human oversight mechanisms designed to prevent automation bias and enable intervention ¹ .	Tri-state risk gating; automated inhibition of autonomous execution for high-risk actions ($R \geq$) ¹ .
EU AI Act: Article 15	System accuracy, robustness, cybersecurity resilience, and protection against exploits ²² .	Dynamic counterfactual perturbation testing and multi-model consensus verification ¹ .
CFPB Circular 2022-03	Prohibition of algorithmic complexity as an excuse for failing to provide adverse action reasons ¹⁵ .	Complete provenance separation of observed enterprise facts from model inferences ¹ .
CFPB Circular 2023-03	Denial notices must accurately reflect the specific non-traditional factors scored ⁷ .	Evidence receipts explicitly capturing missing information and scored behavioral attributes ¹ .
NIST AI RMF (AI 600-1)	Comprehensive Govern, Map, Measure, and Manage functions for autonomous	Integration of Murphy calibration scoring, Cedar policy gates, and runtime

	systems ²² .	risk calculations ¹⁰ .
ISO/IEC 42001 Standard	Controls A.6 and A.9: Continuous AI risk assessment, treatment, and auditability ³⁹ .	Closed-loop consequence tracking, human override telemetry, and cryptographic verification ¹ .

Industry Benchmarking and Functional Gap Analysis

A common engineering misstep is attempting to govern fast decision models using tools designed for generative LLMs or traditional batched machine learning¹. The current tooling landscape divides into three primary approaches: Generative AI Guardrails, Classical ML Observability Platforms, and Enterprise Governance, Risk, and Compliance (GRC) Suites¹. Each exhibits functional limitations when applied to real-time, non-autoregressive decision models¹.

Generative AI Guardrails, such as NeMo Guardrails or Llama Guard, inspect conversational text streams for prompt injection, toxic language, and PII leakage. These tools carry severe latency overheads (often 500 to 3000 milliseconds), making them unusable within sub-second decision paths where Jev operates between 70 and 500 milliseconds². Furthermore, they evaluate unstructured prose rather than enforcing constraints over typed probability distributions, financial limits, or cryptographic schemas¹.

Classical ML Observability Platforms, such as Arize AI or Fiddler AI, focus on asynchronous monitoring of tabular models in batch settings. While skilled at detecting population drift or computing attribution metrics over data warehouses, their architectures are passive and out-of-band¹. They offer no inline enforcement mechanism to intercept, challenge, or inhibit an unauthorized effector action in real time¹.

Enterprise GRC Suites, including IBM watsonx.governance and OneTrust, function as administrative inventory portals for model registration, documentation, and compliance questionnaires¹. These platforms exist outside the software hot path, providing zero real-time visibility into production execution and no technical defense against runtime threshold laundering or context poisoning¹.

Technical Capability	Generative AI Guardrails	Classical ML Observability	Enterprise GRC Platforms	Proposed AI Decision Firewall
Runtime Form Factor	Inline proxy for conversational text streams.	Passive asynchronous sidecar or batch collector.	Administrative web portal and database ¹ .	Inline Policy Enforcement and Decision Point (PEP/PDP) ⁸ .

Hot-Path Latency	500 ms – 3,000 ms (Incompatible with fast decision models) ⁵ .	Out-of-band; zero impact on execution hot path.	N/A (Offline governance platform) ¹ .	2 ms – 15 ms (Microsecond-scale policy engine evaluation) ⁹ .
Enforcement Action	String replacement, token suppression, session termination.	Passive metric alerting and dashboard generation.	Manual workflow reviews and policy publishing ¹ .	Cryptographic capability token issuance (ALLOW / DENY) ¹⁰ .
Policy Specification	Natural-language prompts, Colang, or semantic rules.	Statistical alert thresholds (PSI, KS-test, drift limits).	Human-readable corporate compliance policies ³⁰ .	Formally verified declarative logic (AWS Cedar / Lean) ⁹ .
Audit Artifact	Ephemeral, unverified JSON request and response logs ¹⁶ .	Aggregated feature distributions and statistical summaries.	Model documentation, risk matrices, and compliance spreadsheets ¹⁸ .	Signed, chained decision receipts (JCS, Ed25519, Rekor) ¹⁶ .
Causal Grounding	Self-reported conversational chain-of-thought rationales ¹ .	Post-hoc SHAP and LIME attribution approximations ¹⁵ .	Qualitative design descriptions and stated intent ³⁷ .	Active counterfactual perturbation and evidence provenance ¹ .
Consequence Tracking	None; visibility terminates at token generation.	Partial; measures delayed labels within batch tables.	None; divorced from real-time operational outcomes ¹ .	Persistent linkage: Intent → Choice → Permission → Consequence ⁶ .

Architectural Implementation Roadmap and Recommendations

Integrating the Decision Firewall must follow an incremental path to establish complete operational visibility and policy control before introducing active runtime blocking⁶. This prevents operational disruption while systematically closing execution bypass routes⁶.

The rollout progresses across four distinct operational phases:

In the initial Observation Phase, the firewall operates strictly as an asynchronous shadow listener tapped into production traffic⁶. Input states and typed questions pass directly to the Jev decision model, and actions execute along established pathways⁶. The firewall mirrors this operational traffic, evaluating shadow Cedar policies, calculating theoretical risk scores, and generating draft decision receipts⁶. This phase establishes empirical baseline performance, quantifies hot-path latency impacts, and calibrates risk score thresholds without impeding live workflows⁶.

The second stage, the Active Gating Phase, moves the firewall directly into the execution path between the model and downstream adapters¹. The primary decision model no longer triggers actions directly; instead, it emits a candidate choice to the firewall⁶. The firewall evaluates quantitative risk scores, executes counterfactual sensitivity checks, and processes deterministic Cedar policies¹. If permitted, the gate issues a signed, time-limited capability token to the downstream adapter¹⁰. If the action breaches risk thresholds, the gate inhibits dispatch and routes the transaction to the human review queue¹.

The third stage, the Immutable Receipting Phase, mandates cryptographic proof for all system operations⁸. Downstream execution adapters are reconfigured to reject any command that lacks a cryptographically validated capability token⁶. Every permitted action and rejected proposal generates a canonicalized, Ed25519-signed decision receipt incorporating evidence provenance and declared commercial objectives¹. The head hash of the internal Merkle tree is anchored to a public Sigstore Rekor log, establishing permanent audit trails for regulatory compliance¹⁶.

The final stage, the Closed-Loop Consequence Phase, connects downstream operational outcomes directly back to historical decision receipts¹. The observability hub tracks dispute rates, human overrides, credit defaults, and conversion data against original decision IDs¹. Murphy Brier decompositions and Adaptive Calibration Error calculations run continuously across production data, automatically notifying engineering teams when calibration drift, disparate impact, or incentive laundering emerge¹.

To operationalize this roadmap effectively, engineering teams must execute several key technical directives:

First, embed the AWS Cedar Rust runtime directly into the gateway process using Foreign Function Interfaces (FFI) or WebAssembly (WASM), ensuring that deterministic policy enforcement executes in microseconds without external network dependencies⁹. Second, configure execution adapters to require signed PASETO or JWT capability tokens, ensuring that systems cannot execute actions without explicit, cryptographically bound firewall permission⁶.

Third, implement pre-execution counterfactual checks across all commercially monetized endpoints, automatically stripping commercial variables to detect incentive laundering before dispatch¹. Fourth, structure all audit pipelines around the IETF Acta signed-receipt specification, preserving observed evidence provenance while avoiding the storage of unprotected consumer text⁶. Finally, deploy consequence observability that calculates weekly Murphy reliability scores and monitors disparate impact ratios across protected groups, ensuring long-term legal defensibility and algorithmic fairness¹.

Strategic Outlook and Conclusions

The adoption of constrained, non-autoregressive decision models represents a critical evolution in enterprise automation². By delivering sub-second, typed judgments at scale, engines like TypeSafe AI's Jev will increasingly manage programmatic workflows across modern digital infrastructure². However, placing raw probabilistic outputs directly into autonomous execution loops creates severe operational exposures, including silent bias, incentive laundering, proxy discrimination, and regulatory non-compliance¹.

True enterprise trust cannot be established by relying on vendor claims of internal calibration, nor can it be secured through subjective conversational explanations¹. A calibrated probability describes statistical consistency, not operational correctness or regulatory compliance¹.

By deploying an external Decision Firewall, organizations establish an enforceable, mathematically defensible control plane around autonomous systems¹. This architecture ensures that decision models are restricted to their proper role: evaluating state and proposing actions⁶. Permitting those actions to execute requires verifiable evidence provenance, deterministic policy validation, cryptographic receipts, and ongoing accountability to real-world outcomes⁶. Integrating this governance layer transforms fast probabilistic models from unmonitored liabilities into resilient, compliant, and auditable enterprise assets¹.

Works cited

1. info.md
2. Jev (AI model) - Wikipedia, [https://en.wikipedia.org/wiki/Jev_\(AI_model\)](https://en.wikipedia.org/wiki/Jev_(AI_model))
3. Jev Explained: Typesafe AI's Non-Autoregressive System-1 Model, <https://www.mindstudio.ai/blog/jev-system-one-model-launch>
4. What Is Jev? TypeSafe AI's System One Model Explained - You.com, <https://you.com/resources/what-is-jev>
5. What is Jev (2026)? TypeSafe AI's System One model | DigitalOcean, <https://www.digitalocean.com/resources/articles/what-is-jev>
6. info-2.md
7. CFPB Issues Additional Guidance on Use of AI in Credit Underwriting, <https://www.bradley.com/insights/publications/2023/10/cfpb-issues-additional-guidance-on-use-of-ai-in-credit-underwriting>
8. Institutional Attestation as a Governance Model for Autonomous AI, <https://arxiv.org/html/2606.26298v1>
9. Authorization Management: Zero-Trust Policy for the Agentic,

- <https://www.gravitee.io/blog/authorization-management-zero-trust-policy-for-the-agent-enterprise>
10. OpenLeash vs Cedar for AI Agent Authorization, <https://openleash.ai/concepts/openleash-vs-cedar/>
 11. Cedar Language, <https://cedarpolicy.com/>
 12. Cedar: the access control building block that's perfectly timed for AI agents, <https://www.thestack.technology/cedar-the-access-control-building-block-thats-perfectly-timed-for-ai-agents/>
 13. Understanding Cedar policies - Amazon Bedrock AgentCore, <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy-understanding-cedar.html>
 14. Secure AI agents with Policy in Amazon Bedrock AgentCore - AWS, <https://aws.amazon.com/blogs/machine-learning/secure-ai-agents-with-policy-in-amazon-bedrock-agentcore/>
 15. CFPB Circular 2022-03, https://files.consumerfinance.gov/f/documents/cfpb_2022-03_circular_2022-05.pdf
 16. draft-farley-acta-signed-receipts-03 - IETF Datatracker, <https://datatracker.ietf.org/doc/draft-farley-acta-signed-receipts/>
 17. Provenance - SLSA.dev, <https://slsa.dev/spec/v1.0/provenance>
 18. Attestation across the AI Supply Chain - Data Leverage Digital Garden, <https://nickmvincent.github.io/long-posts/attestation-across-the-ai-supply-chain.html>
 19. Evidence & Verification | Trinitite Docs, <https://docs.trinitite.ai/docs/products/evidence-verification/>
 20. Claims and verification - AxioRank, <https://axiorank.com/docs/claims>
 21. Securing AI Agents with Cryptographic Receipts, <https://microsoft.github.io/ai-agents-for-beginners/18-securing-ai-agents/>
 22. NIST AI RMF Explained: AI Governance FAQs for Leaders | Cycore, <https://www.cycoresecure.com/blogs/nist-ai-rmf-explained-15-faqs-ai-leader-needs-answered>
 23. Unpack the notation used in Wikipedia's decomposition of the Brier, <https://stats.stackexchange.com/questions/631333/unpack-the-notation-used-in-wikipedias-decomposition-of-the-brier-score>
 24. How does the Brier Score break down to (Reliability - Cross Validated, <https://stats.stackexchange.com/questions/597679/how-does-the-brier-score-break-down-to-reliability-resolution-uncertainty>
 25. Scoring Rules: Brier Score with Decomposition Calculator - MetricGate, <https://metricgate.com/docs/scoring-rules-brier/>
 26. Understanding Model Calibration: A Gentle Introduction & Visual, <https://towardsdatascience.com/understanding-model-calibration-a-gentle-introduction-visual-exploration/>
 27. Calibration of Deep Learning Classification Models in fNIRS, <https://www.techrxiv.org/doi/pdf/10.36227/techrxiv.170861971.16301364>
 28. Why is binning in Expected Calibration Error done the way it is?, <https://stats.stackexchange.com/questions/544835/why-is-binning-in-expected-cali>

[bration-error-done-the-way-it-is](#)

29. CFPB Circular 2022-03: Complex Lending Algorithms Cannot, <https://www.gtlaw.com/en/insights/2022/6/cfpb-circular-2022-03-complex-lending-algorithms-adverse-credit-determination>
30. Learn All About the NIST AI Risk Management Framework - Optro, <https://optro.ai/blog/what-is-the-nist-ai-risk-management-framework>
31. What Is the NIST AI Risk Management Framework? - SAI360, <https://www.sai360.com/resources/blog/what-is-the-nist-ai-risk-management-framework>
32. EU AI Act — Aligned by Design | Graql, <https://graql.com/eu-ai-act>
33. NIST AI RMF Explained: Functions, Playbook & Compliance | Alation, <https://www.alation.com/blog/nist-ai-rmf/>
34. Braun CFPB Circular Creditors Use of Artificial Intelligence, <https://www.adamsandrees.com/insights/braun-cfpb-circular-creditors-use-of-artificial-intelligence>
35. Latest CFPB Circular Regarding Artificial Intelligence and Credit, <https://www.sullcrom.com/insights/memo/2023/September/CFPB-Issues-Circular-Regarding-Artificial-Intelligence-and-Credit-Decisions>
36. Back to Basics, Continued — CFPB Circular 2022-03; Adverse, <https://www.dentons.com/en/insights/newsletters/2022/june/8/consumer-finance-report/back-to-basics-continued-cfpb-circular-2022-03>
37. NIST AI RMF Implementation: 8-Step Guide + 20-Week Roadmap, <https://neuraltrust.ai/blog/nist-ai-rmf-implementation-guide>
38. NIST AI 600-1: The 12 Generative AI Risk Categories - DSALTA, <https://www.dsalta.com/resources/ai-compliance/nist-ai-600-1-generative-ai-profile-12-risks-explained>
39. What Is ISO 42001? The AI Management System Standard Explained, <https://www.snowflake.com/en/artificial-intelligence/ai-governance/iso-42001/>
40. Your guide to ISO 42001 controls - Vanta, <https://www.vanta.com/collection/iso-42001/iso-42001-controls>
41. Your Guide to ISO 42001 Controls for AI Governance - Sprinto, <https://sprinto.com/iso-42001/controls/>